

GPU Accelerated Conformal Prediction

Anonymous

Submitted for the Degree of Master of Science in
Artificial Intelligence



Department of Computer Science
Royal Holloway University of London
Egham, Surrey TW20 0EX, UK

August 28, 2025

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 11833

Student Name: Anonymous

Date of Submission: 28.08.2025

Abstract

Conformal Prediction is a recent advancement, that aims to provide a quantification of uncertainty to a predictive model. This report investigates the efficiency of using GPU Acceleration to perform conformal prediction on a K-Nearest Neighbour model using CUDA. The overall objective is to compare the difference in performance of the model with the CPU and see how the augmentation of the number of features effects the performance of the GPU model.

This is approached by utilising the power of CUDA and PyTorch in Python; two scripts are produced for both and CPU and the GPU with their own results. Comparisons can be made between the runtimes and the performance of the models by the given mean squared errors of the models, which are all documented throughout the results section of the report.

Results demonstrate that the GPU models generally achieve a coverage probability exceeding approximately 95%, indicating reliable prediction performance. Non-conformity scores, which measure the uncertainty generated by conformal prediction, reveal that the predictions are accurate and that their distribution remains stable across different feature sets.

Attached in the appendix are guides to running both scripts; these can be used to further develop the efficiency of the models or be used to perform conformal prediction on alternate datasets with a few tweaks to the selection of features.

Contents

1	Introduction	1
1.1	Background	1
1.2	Objectives of the Study	2
2	Literature Review	3
2.1	K-Nearest Neighbours Regression in Machine Learning	3
2.2	Overview of Conformal Prediction	5
2.3	Importance of Feature Selection	7
2.4	Computer Hardware and Parallelisation	8
2.4.1	Parallelisation and CUDA-Based GPU Acceleration	9
2.4.2	CPU Parallelism and Multi-Core Architectures	9
2.4.3	GPU Acceleration and CUDA Framework	10
2.4.4	Performance Optimisation and Advanced Techniques	10
2.4.5	Challenges and Future Directions	11
2.5	Summary of Literature (Key definitions)	11
3	Methodology	13
3.1	Broad goals and expectations	13
3.2	Dataset Description	13
3.3	Data Handling in Python	14
3.4	Extracted features	14
3.5	Data Pre-processing	14
3.5.1	Encoding Categorical Features	15
3.5.2	Standardisation	16
3.6	Model Description	16
3.6.1	K-Nearest Neighbours (KNN)	16
3.6.2	Conformal Prediction Methodology	18
3.7	Parallelisation using CPU	19
3.8	GPU Acceleration with PyTorch	21
3.8.1	Distance Calculation	22
3.8.2	Computation Process	23
3.8.3	Batching, GPU Optimisation, and Handling Large Datasets	23
4	Implementation	25
4.1	Setting up the Environment	25
4.1.1	Installing VSCode	25

4.1.2	Setting up CUDA	25
4.1.3	Hardware and Software Used	26
4.2	Data Pre-processing Implementation	27
4.3	KNN Prediction	28
4.3.1	PyTorch Implementation	28
4.3.2	NumPy Implementation	29
4.4	Conformal Prediction Interval Calculation	30
4.5	GPU and CPU Performance	32
4.6	Evaluation metrics for the algorithm	32
5	Results	34
5.1	MSE, RMSE and coverage probability for Different Feature Sets on GPU	34
5.1.1	CPU MSE/RMSE	34
5.2	Graphs of predictions for each feature set	35
5.2.1	CPU Acceleration	35
5.2.2	GPU Acceleration	36
5.3	CDF of Non-Conformity Scores for GPU models	39
5.4	Performance Analysis	40
5.4.1	Runtime Comparison between GPU and CPU	40
6	Discussion	41
6.1	Interpretation of Results	41
6.1.1	Summary of Results	43
6.2	Practical Implications	44
6.3	Limitations of the Study and Future Work	44
6.4	Personal Assessment	45
	References	47
A	A: How to run the Python scripts	49
A.1	Prerequisites	49
A.2	Running the Scripts	49

1 Introduction

1.1 Background

In recent years, machine learning has seen continuous improvements, particularly in predictive modelling and uncertainty quantification. A significant advancement is conformal prediction, a method that provides valid and reliable measures of calculating prediction uncertainty [9]. Unlike traditional methods that offer point estimates, conformal prediction generates prediction intervals, mathematically ensuring that the true value will lie within the interval with a specified probability. This is especially valuable in fields where understanding the uncertainty of predictions is as crucial as the predictions themselves.

At the same time, the exponential growth in data and model complexity has driven the need for more powerful computational resources. Modern GPUs, with their ability to process tasks in parallel, offer substantial speed-up in machine learning algorithms [21]. Their capabilities allow for faster training and predictions, enabling the efficient handling of larger datasets and more complex models. As a result, GPU acceleration has become a vital component of machine learning workflows, opening new possibilities for both research and real-world applications.

This dissertation aims to combine these two important developments: conformal prediction and GPU acceleration. Specifically, this dissertation develops a GPU-accelerated model for car price prediction that incorporates conformal prediction techniques. The objective is not only to create an accurate predictive model but also to produce reliable measures of uncertainty through prediction intervals. Conformal prediction enhances the model's ability to quantify uncertainty, while GPU acceleration ensures scalability and improved performance.

Although the study focuses on car price prediction using multiple features in a dataset, its methodologies are applicable beyond this scope. The combined use of conformal prediction and GPU acceleration is relevant for any domain that requires accurate and computationally efficient predictive modelling, demonstrating the potential of this approach for a wide range of machine learning applications.

1.2 Objectives of the Study

The primary objective of this study is to develop a GPU-accelerated conformal prediction model for car price prediction, providing accurate predictions with reliable uncertainty estimates. The specific objectives are:

1. **ML Algorithm:** Decide and explain my choice of a machine learning algorithm suitable for this project.
2. **Conformal Prediction:** Understand and explain Conformal Prediction and why it is important.
3. **GPU Acceleration:** Explain the relevance and importance of GPU-Acceleration.
4. **Implementation:** Design and implement a Machine Learning model that generates predictions and intervals for my dataset, using CPU and GPU.
5. **Evaluate Performance:** Evaluate the impact of running the algorithm on CPU Vs GPU.
6. **Evaluate Feature Selection:** Evaluate the impact of changing the selected features on the model.
7. **Documentation:** Thoroughly document methods, results, and insights to contribute to future studies.

2 Literature Review

2.1 K-Nearest Neighbours Regression in Machine Learning

Machine Learning (ML) is a sub-field of Artificial Intelligence (AI) that focuses on the development of algorithms capable of learning patterns from data and making predictions or decisions without being explicitly programmed for each task. According to [19], the essence of machine learning can be captured by the following definition: "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T , as measured by P , improves with experience E ." In simpler terms, ML algorithms use historical data as input to predict or classify unseen instances by recognising underlying patterns.

One of the fundamental algorithms in ML is the K-Nearest Neighbours (KNN) method. KNN is a type of non-parametric learning, where the model does not explicitly learn a function from the training data but rather memorises the training dataset itself. For any given input data point, the KNN algorithm identifies the k closest training examples in the feature space and makes predictions based on these nearest neighbours.

We can define this mathematically; given a training dataset,

$$(\mathcal{Z} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\},$$

where $x_i \in \mathcal{X}$ represents a feature vector and $y_i \in \mathcal{Y}$ denotes the corresponding label. The feature vector x_i represents multiple features which describes the instance. For example, a dataset that relates to house prices may include features describing the size of a house, the number of rooms available in it, and the age of the property. The label, y , is then the outcome associated with the feature vector, where y would be the price for the example of a house dataset.

The KNN prediction for a new point x_{n+1} is computed as follows:

$$y_{n+1} = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x_{n+1})} y_i$$

Here, $\mathcal{N}_k(x_{n+1})$ represents the set of indices corresponding to the k nearest neighbours of x_{n+1} , and the prediction y_{n+1} is the average of the labels y_i of these nearest neighbours. The distance metric used to determine the nearest neighbours is typically the Euclidean distance, defined as:

$$d(x_i, x_j) = \sqrt{\sum_{l=1}^p (x_{il} - x_{jl})^2}$$

where x_{il} and x_{jl} are the l -th features of the data points x_i and x_j respectively, and p is the total number of features.

We can model a quick algorithm for calculating euclidean distance, like followed:

Algorithm 1 Calculate Euclidean Distance

```

1: procedure EUCLIDEANDISTANCE(p, q)
2:   distance  $\leftarrow$  0
3:   for  $i = 1$  to  $n$  do
4:     difference  $\leftarrow$  p[ $i$ ] - q[ $i$ ]
5:     distance  $\leftarrow$  distance + difference2
6:   end for
7:   distance  $\leftarrow$   $\sqrt{\text{distance}}$ 
8:   return distance
9: end procedure

```

Putting everything together, below is design of a KNN algorithm, very similar to the one found at [6]:

Algorithm 2 K-Nearest Neighbours Regression

```

1: Input: Training dataset ( $X_{train}, y_{train}$ ), Test dataset  $X_{test}$ , Number of
   neighbours  $k$ 
2: Output: Predicted values for  $X_{test}$ 
3: Initialise an empty list  $y_{pred}$ 
4: for each test sample  $x_{test}$  in  $X_{test}$  do
5:   Compute distances between  $x_{test}$  and all samples in  $X_{train}$ 
6:   Sort distances and find the indices of the  $k$  smallest distances
7:   Retrieve the  $k$  nearest neighbour target values from  $y_{train}$  using these
   indices
8:   Compute the average of these  $k$  target values
9:   Append the computed average to  $y_{pred}$ 
10: end for
11: Return  $y_{pred}$ 

```

The choice of k , the number of neighbours, significantly impacts the

performance of the KNN algorithm: a smaller k may lead to overfitting, while a larger k can result in underfitting. Underfitted models are prone to high bias; they give inaccurate results for both the training data and test set. Whereas overfitted models experience high variance; they give accurate results for the training set but less accurate results for the test set. The non-parametric nature of KNN is discussed in more detail in [3], which provides a comprehensive introduction to the method and its variations.

In the context of regression models, including KNN regression, evaluating the accuracy of predictions is crucial. Two widely used metrics for this purpose are the Mean Squared Error (MSE) and the Root Mean Squared Error (RMSE). The MSE measures the average of the squares of the errors, also described as the average squared difference between the estimated values and the actual value:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where y_i denotes the true value, $\hat{y}_i$ represents the predicted value, and n is the number of observations. MSE provides a measure of the quality of an estimator or a predictor. The squaring of the differences amplifies the effect of outliers, making MSE a more sensitive metric to large errors.

RMSE is simply the square root of the MSE, giving it the same units as the target variable, which can make interpretation more straightforward:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

RMSE is particularly useful in regression problems as it provides a clear indication of how far the predictions are from the actual values on average. The relevance of MSE and RMSE in forecasting and regression tasks is extensively discussed in [13].

2.2 Overview of Conformal Prediction

Conformal prediction is a recent statistical framework that aims at the reliable quantification of uncertainty in predictive modelling. The approach, originally proposed by Vladimir Vovk, Alex Gammerman, and Glenn Shafer [9], allows the construction of prediction intervals or sets at any user-defined confidence level, typically 95%, based on previous experience. These prediction intervals provide a range within which we expect the true response to

fall, offering a measure of uncertainty around point estimates made by the model.

A key aspect of conformal prediction is ensuring the desired coverage probability, which refers to the proportion of times the prediction set contains the true outcome [14]. For example, with a 95% confidence level, we expect the prediction set to include the true value 95% of the time across many predictions. Consequentially, a high coverage probability means the model will be well-calibrated and will ensure reliability in the quantification of uncertainty.

The framework can be described as follows.

$$\mathcal{Z} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

as before represents a training dataset, where each pair (x_i, y_i) consists of a feature vector $x_i \in \mathcal{X}$ and a corresponding label or response variable $y_i \in \mathcal{Y}$.

The objective of conformal prediction is to predict the label y_{n+1} for a new feature vector x_{n+1} , which the model has not seen before. Rather than providing a single point estimate for y_{n+1} , conformal prediction seeks to construct a prediction set $C(x_{n+1})$ that contains all plausible values for y_{n+1} based on the information from the training dataset $\mathcal{Z}$.

This prediction set $C(x_{n+1})$ is constructed in such a way that it includes the true value y_{n+1} with a high probability, referred to as the confidence level $1 - \alpha$. For instance, if $1 - \alpha = 0.95$, then the prediction set $C(x_{n+1})$ is designed so that it contains the true value y_{n+1} 95% of the time. The confidence level allows for controlling how broad or conservative the prediction set is.

At its core, conformal prediction is built on the idea of non-conformity measures. These measures quantify how unusual or non-conforming a new observation is, relative to a set of previously observed data points. For example, in the context of regression, a non-conformity measure might be the absolute error between the observed and predicted values. For classification, it might involve the probability assigned to the predicted class by a machine learning model.

The non-conformity measure α is a function $\alpha : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$ that assigns a real number α_i to each example (x_i, y_i) in the training set, representing the degree to which the example does not conform to the rest of the training set. A typical non-conformity measure might be based on the error or residual of a predictive model. For instance, in regression, a simple non-conformity measure could be the absolute difference between the predicted and actual

values:

$$\alpha_i = |y_i - \hat{y}_i|,$$

where $\hat{y}_i$ is the predicted value for x_i given by the model. A higher value of α_i indicates that (x_i, y_i) is more non-conforming or unusual compared to the model's predictions. This α_i that we discussed here is also sometimes referred to as a Z score.

To implement conformal prediction, we will make some modifications to the standard approach of splitting the data used in a KNN model. Instead of dividing the data into two sets, namely training and testing, we add one more, the validation set. The validation set allows for model performance assessment and computation of the non-conformity scores. KNN trains using the training set and makes some predictions on the validation set. This will have the differences between the model's predictions and the real values in the validation set as the non-conformity scores. Having obtained the non-conformity scores, a given quantile of these scores is then determined, which would be the 95th percentile to give us a 95% confidence interval. This quantile serves as a threshold that defines the bounds of our prediction intervals. In such cases, the quantile is added and subtracted from the predicted values so that the prediction intervals in KNN characterise the uncertainty associated with the model's prediction.

2.3 Importance of Feature Selection

Among others, the important goals of feature selection, according to Guyon and Elisseeff, 2003 [11], are the following. First, the model complexity can be reduced by excluding those features that are irrelevant or redundant; basically those features that do not help the predictive power of the model. Feature selection reduces the input variables within the model, hence "streamlining" the model for quicker times in training and evaluation. This will become all the more important in high-dimensional datasets, since the existence of irrelevant features will force the model to learn noise instead of real underlying patterns in the data.

Second, feature selection also strengthens the generalisation ability for machine learning algorithms. One basic problem common in machine learning often faced by models is what is termed the "curse of dimensionality": performance deteriorates as the number of features increases. It can generalise better to unseen data by considering a subset of the most important features, hence improving predictive accuracy on test sets. This is particularly true for algorithms such as k-nearest neighbours, where distances between data points are hard to compute accurately in high-dimensional

spaces. Another main advantage of feature selection noted by [11] concerns interpretability. Models that use fewer features are more intuitive and easier to explain. This alone makes them more necessary, as in domains like health, finance, and engineering, stakeholders many times must understand what output a machine learning model is providing. For example, in the case of predicting vehicle sales, only a few key elements would be considered that are highly relevant to the task, like condition, mileage, and brand, for the model to allow the analysts and the decision-makers to have a better understanding of what is going on with the predictions made without extraneous information.

2.4 Computer Hardware and Parallelisation

In most cases, the performance of machine learning algorithms is highly dependent on the underlying hardware. The Central Processing Unit (CPU) is crucial for running these algorithms, as it handles the majority of general-purpose calculations. Modern CPUs are equipped with multiple cores, each functioning as an independent processing unit capable of managing a variety of instructions simultaneously. This multi-core architecture is particularly beneficial when dealing with computationally demanding tasks, such as those involving large datasets, as it enables efficient execution through parallel processing [12]. According to Hennessy and Patterson (2019), the evolution of CPU architectures towards multi-core designs has been instrumental in improving the performance of data-intensive applications by distributing workloads across multiple processing units.

Graphics Processing Units (GPUs) serve as complementary hardware to CPUs, especially for tasks that require substantial mathematical computations. GPUs are designed with thousands of smaller, simpler cores optimised for high-throughput parallel processing. This architecture allows tasks to be broken down into smaller units that can be processed simultaneously across multiple cores. NVIDIA’s CUDA (Compute Unified Device Architecture) is particularly effective in this context, as it provides a parallel computing platform that allows developers to harness the full potential of GPU resources. CUDA facilitates the execution of tasks such as matrix operations, which are critical in training deep learning models, by enabling parallel computations across thousands of cores [10, 16]. Goodfellow et al. (2016) specifically highlight the role of GPUs in accelerating deep learning processes, noting that CUDA’s ability to handle large-scale matrix multiplications significantly enhances the efficiency of these operations.

Additional key hardware components include Random Access Memory

(RAM) and Solid State Drives (SSDs). RAM provides temporary, volatile storage that is essential for holding datasets during processing, allowing for quick access and manipulation of data. SSDs, on the other hand, offer permanent, non-volatile storage and are crucial for the fast recall of data, which is especially important in large-scale machine learning applications [7].

2.4.1 Parallelisation and CUDA-Based GPU Acceleration

When dealing with extensive datasets, such as those exceeding 50,000 entries, traditional sequential processing methods often become inadequate due to their inherent limitations in handling large scale computations. This has led to the adoption of advanced techniques like parallelisation to mitigate runtime bottlenecks and enhance computational capabilities.

Parallelisation is the process of decomposing a task into smaller, independent sub-tasks that can be executed concurrently across multiple processing units. This approach takes advantage of the multi-core architecture of modern CPUs and the massively parallel capabilities of GPUs, leading to significant improvements in computational efficiency and reductions in execution time [12, 4]. As noted by Amdahl (1967), the potential speedup of a computational process through parallelisation is limited by the proportion of the task that must be executed sequentially. However, with careful optimisation, the majority of tasks can be parallelised, leading to substantial gains in performance.

2.4.2 CPU Parallelism and Multi-Core Architectures

CPUs remain central to executing a wide range of computational tasks, including those associated with machine learning algorithms. Contemporary CPUs, equipped with multiple cores, enable the parallel execution of separate threads, enhancing throughput and reducing latency. Hennessy and Patterson (2019) discuss how this multi-core architecture allows for the division of complex computations into parallel tasks, which is particularly effective in reducing the time required for tasks such as training deep neural networks. Operations like forward propagation, backpropagation, and gradient descent, which involve extensive matrix multiplications and tensor operations, can be significantly accelerated by distributing these computations across multiple CPU cores [12].

2.4.3 GPU Acceleration and CUDA Framework

A Graphics Processing Unit (GPU) is a specialised hardware component originally designed for rendering images and performing graphical computations. While CPUs are versatile, GPUs can be specifically optimised for high-throughput parallel processing. We mentioned earlier the CUDA framework developed by NVIDIA is a key enabler of this parallel processing capability. CUDA provides developers with the tools to write programs that properly utilise the parallel processing power of GPUs, thereby accelerating computationally intensive tasks [16, 21].

CUDA-optimised libraries, such as cuBLAS and cuDNN, are particularly valuable in machine learning applications. These libraries offer efficient implementations of common linear algebra operations and deep learning primitives, respectively, allowing for significant speedups in both training and inference processes. This capability is crucial for handling large-scale models and datasets, as it enables the execution of complex operations in a fraction of the time required by traditional CPU-based methods [22].

2.4.4 Performance Optimisation and Advanced Techniques

The advantages of parallelisation and CUDA-based GPU acceleration are substantial. By distributing computational workloads across multiple GPU cores, tasks such as clustering, dimensionality reduction, and predictive analytics can be executed more rapidly. This parallel processing capability is particularly beneficial for real-time applications and iterative model updates [26].

Performance optimisation involves several strategies aimed at maximising the efficiency of parallel computations. Techniques such as load balancing, memory management, and minimising data transfer overhead are critical to achieving optimal performance on GPUs [17]. Load balancing ensures that all cores are effectively utilised, while memory management optimises memory access patterns. Reducing data transfer between the host and the device memory is essential to mitigate bottlenecks that can hinder overall performance.

In heterogeneous computing environments, which combine CPUs and GPUs, there are additional opportunities for performance optimisation. In these setups, CPUs manage tasks that require complex control logic and frequent branching, while GPUs handle computations which benefit greatly from being run in parallel. This hybrid approach leverages the strengths of both types of processors to achieve superior overall efficiency [20].

2.4.5 Challenges and Future Directions

Despite the benefits, parallelisation and CUDA-based acceleration of GPU raise a number of challenges. As a matter of fact, while the development of effective parallel algorithms is a difficult task, it needs to consider data dependencies, synchronisation mechanisms, and load distribution carefully to avoid degradation in performance. [12].

Looking ahead, new hardware technologies such as TPUs (Tensor Processing Units) and FPGA (Field-Programmable Gate Array) accelerators hold great promise for even further advances in parallel computing. TPUs, designed by Google, are specialised for tensor processing and can achieve spectacular speedups for certain machine learning tasks, especially those which rely on deep learning. FPGAs provide custom hardware acceleration, thus enabling performance optimisation for specific algorithms[15, 24].

2.5 Summary of Literature (Key definitions)

- **Machine Learning (ML):** Subset of AI, focuses on developing algorithms capable of learning patterns from data and making predictions without explicit programming for each task.
- **K-Nearest Neighbours (KNN):** A non-parametric learning algorithm where predictions for new data points are made based on the average labels of the k -nearest neighbours in the training data, determined by a distance metric like Euclidean distance.
- **Mean Squared Error (MSE):** A metric that measures the average squared difference between the predicted and actual values in a regression model, amplifying the impact of large errors.
- **Root Mean Squared Error (RMSE):** The square root of MSE.
- **Conformal Prediction:** A statistical framework that constructs prediction intervals with a specified confidence level.
- **Non-conformity Measure:** A quantifier used in conformal prediction to determine how unusual a new observation is, based on the error between observed and predicted values.
- **Feature Selection:** The process of selecting the most relevant variables in a dataset, reducing the model's complexity and improving its generalisation ability, often leading to faster training and evaluation.

- **Parallelisation:** A computational technique that divides tasks into smaller subtasks that can be executed simultaneously across multiple processing units, improving performance and efficiency.
- **GPU Acceleration:** The use of a GPU to accelerate computational tasks by performing parallel operations, significantly reducing the run-time of algorithms

3 Methodology

3.1 Broad goals and expectations

Given our objectives we laid out in section 1, for our practical implementation we aim to generate two algorithms. One for the CPU and one for the GPU.

Both algorithms should run a KNN linear regression analysis on the dataset, using their respective method of parallelisation, to produce a plot showing the predicted values and the true values for each prediction. These values should also include the intervals generated by conformal prediction to provide a level of confidence for the overall predictions. Furthermore, the non-conformity scores should be stored such that we can generate a cumulative distribution graph of the scores to see how reasonable our predictions will be.

We will attempt to augment the number of features that each algorithm is run for, to allow us to discuss the importance of feature selection for predictive modelling. I predict the models to be more accurate with the addition of more features, at the cost of runtime.

3.2 Dataset Description

When choosing a dataset, it is important that the data is suitable for the type of machine learning algorithm being applied. Since this study employs K-Nearest Neighbour (KNN) for regression, a dataset that provides a range of numerical outcomes based on a set of input features is appropriate. House prices were mentioned earlier, and similarly, this study evaluates car prices based on features such as mileage, year, and condition. The dataset used contains 558,837 data points and 14 features, with the target variable being the selling price of the car. The dataset includes attributes like year, make, model, odometer reading, condition, and other relevant factors for predicting vehicle prices. This dataset is publicly available at [5], and our selected features for evaluation are discussed in section 3.4.

It is important to note that whilst this study is using the car prices dataset, the methods used in regards to conformal prediction and parallelisation can be done on any dataset provided an appropriate machine learning algorithm is used in conjunction.

3.3 Data Handling in Python

Most of the handling of data is done quite efficiently in Python by libraries such as pandas, thereby making data manipulation quite straightforward. CSV files are easily imported through pandas for use. The `read_csv()` function is used for reading a CSV file into a DataFrame.

3.4 Extracted features

Before we begin processing our data, we need to select the features we are extracting to make the model. A brief description of the selected features have been provided below, with a tag whether they are a numerical or categorical feature.

- **Odometer** (Numerical): The total mileage of the vehicle.
- **MMR** (Numerical): Manheim Market Report, an estimate of car value based on auction data.
- **Condition** (Numerical): The physical state of the car, typically rated on a scale.
- **Year** (Numerical): The manufacturing year of the vehicle.
- **Make** (Categorical): The manufacturer or brand of the car (e.g., Ford, Toyota).
- **Model** (Categorical): The specific model of the car (e.g., Mustang, Corolla).
- **Body** (Categorical): The body type of the car (e.g., sedan, SUV).

3.5 Data Pre-processing

Data pre-processing is one of the most important steps when starting any machine learning model, and it involves making raw data ready for analysis. This includes data cleaning for the handling of missing or inconsistent entries, transformation of a categorical variable into numerical form, and scaling features so all of them will be within some standard range. Effective pre-processing is essential for machine learning algorithms to perform to the best by removing noise and irregularities, improving convergence, making sure that the model is not biased due to differences in scale or representation. The most sophisticated models can become inaccurate predictors of results due to poor quality data in the absence of proper pre-processing.

Two libraries to help with data pre-processing are `Pandas` and `sklearn.preprocessing`, and documentation for these libraries can be found at [1] and [2] respectively.

3.5.1 Encoding Categorical Features

Most algorithms in machine learning cannot process categorical data directly; they need numerical input. Some examples are below:

- Gender: Male, Female
- Car body: Saloon, SUV, Hatchback
- Country: UK, Spain, Germany

Therefore, there should be some encoding for categorical variables into a numerical format before training. Two of the most popular ways for encoding are Label Encoding and One-Hot Encoding.

Label Encoding assigns a different integer to each category in a column. It best serves in ordinal data, in which categories are intrinsically ordered. For nominal or non-ordinal data, One-Hot Encoding might be suitable, since it creates binary columns for every category and indicates the presence or otherwise for each value.

In Python, the `LabelEncoder` and `OneHotEncoder` classes from `sklearn.preprocessing` are commonly used. `LabelEncoder` can transform a column like "Red", "Green", "Blue" into [0, 1, 2], while `OneHotEncoder` converts each category into a new binary column. Below shows both methods, but `LabelEncoder` would be the most simple method for encoding categorical data in the cars data set.

```
1 # Label Encoding in Python
2 from sklearn.preprocessing import LabelEncoder
3 encoder = LabelEncoder()
4 df['category_column'] = ...
5 ...encoder.fit_transform(df['category_column'])
6
7 # One-Hot Encoding in Python
8 df = pd.get_dummies(df, columns=['category_column'], ...
9 ...drop_first=True)
```

3.5.2 Standardisation

Standardisation is one of the important steps in data pre-processing that makes all the numerical features on the same scale. Most machine learning algorithms, such as K-Nearest Neighbors and gradient-based methods, are sensitive to the scale of input features. Without standardisation, features with large values will dominate the model learning process, thus producing biased predictions.

Standardisation techniques typically include scaling features to have a mean of 0, and a standard deviation of 1. The mathematical formula for standardising a feature is given by:

$$z_i = \frac{x_i - \mu}{\sigma}$$

Where: z_i is the standardised value of the feature, x_i is the original value of the feature, μ is the mean of the feature and σ is the standard deviation of the feature.

This can be done using class `StandardScaler` from `sklearn.preprocessing`. The method `fit_transform` will standardise the training data, and `transform` is used for test and validation datasets, so that the features are consistently scaled.

Given that we have our training, testing and evaluation (for conformal prediction) sets, we can standardise them in Python like so:

```
1      # Standardisation in Python
2      from sklearn.preprocessing import StandardScaler
3
4      scaler = StandardScaler()
5      X_train_scaled = scaler.fit_transform(X_train)
6      X_test_scaled = scaler.transform(X_test)
7      X_val_scaled = scaler.transform(X_val)
8
```

3.6 Model Description

3.6.1 K-Nearest Neighbours (KNN)

K-Nearest Neighbours is a strong choice for car price prediction because of its inherent capacity to capture the hidden complexities of multi-dimensional feature spaces, with no explicit assumption needs to be made on the underlying distribution in the data. The prices of cars are affected by continuous

variables, such as mileage and year, and condition, but also categorical variables like make, model, and transmission. On this count, KNN is particularly very good due to the fact that it is non-parametric. It models complicated, nonlinear interactions between variables without explicit specification of a fixed mathematical relationship between the inputs and outputs. This distance-based approach of KNN fits quite well in the idea of car price prediction. More obviously, the cars that are "near" in the feature space, those with similar features, are likely to have similar prices. For example, two cars having the same make and model year but having different mileages are close in feature space, and their prices will also be quite close.

We can write a very simple KNN algorithm using Python and the `sklearn.neighbors` library. The class `KNeighborsRegressor` helps make this process seamless.

After loading in and cleaning the dataset, we must first split the set into a training and test set. This can be simply done by the following line:

```
X_train, X_test, y_train, y_test...  
...= train_test_split(X, y, test_size=0.2, random_state=12)
```

This line of code splits the dataset into two parts: 80% of the data will go to the training set and the remaining 20% into the test set. It is common to have the training dataset, that is, `X_train`, `y_train`, used for the training of machine learning models and the test dataset, that is, `X_test`, `y_test`, for evaluating the performance of the trained model on unseen data. In this context, `X` represents the input features (such as odometer, MMR, condition, etc.), while `y` represents the target variable, which in this case is the car's selling price. The notation `X` and `y` is widely used in machine learning, where `X` typically refers to the input data matrix, and the use of `y` is conventional in machine learning to represent a 1D vector or an array of target values, where each entry corresponds to the target value for a data point in `X`. What `random_state=12` does is ensure the result is reproducible; that means you will have exactly the same split every time you run your code. The 12 can be any integer, as it represents a seed to reproduce the results.

After the data is split, we can run the standardisation techniques. Then we use the `KNeighborsRegressor` class mentioned above to run the KNN algorithm.

```
1  # Fit a KNN regressor  
2  knn = KNeighborsRegressor(n_neighbors=5)
```

```

3     knn.fit(X_train_scaled, y_train)
4
5     # Predict and evaluate
6     y_pred = knn.predict(X_test_scaled)

```

Alternatively, we can use `numpy` to make our own prediction function. Based off how we know a KNN algorithm works, we can write it in Python like so:

```

1  import numpy as np
2
3  def knn_predict(X_train, y_train, X_test, k):
4      y_pred = []
5      for test_point in X_test:
6          # Compute distances between the test point and all train
           ↪ points
7          distances = np.linalg.norm(X_train - test_point, axis=1)
8          # Get the indices of the k nearest neighbours
9          knn_indices = np.argsort(distances)[:k]
10         # Predict by averaging the target values of the nearest
           ↪ neighbours
11         y_pred.append(np.mean(y_train[knn_indices]))
12     return np.array(y_pred)

```

For future sections that require the use of a KNN predictor, I will be referencing this `knn_predict` function.

3.6.2 Conformal Prediction Methodology

Conformal prediction can be implemented via Python by first altering how we split our data. Instead of 2 splits, we make 3, with the addition of a validation set. K-Nearest Neighbors makes predictions based on the validation set. The nonconformity scores are then calculated. A quantile of these scores, such as the 95th percentile, is used as a threshold to define prediction intervals. When KNN generates the predictions, the quantile derived from the validation scores is added and subtracted to form the prediction intervals.

```

1  # Compute nonconformity scores (absolute differences) on
   ↪ validation set
2  y_val_pred = knn_predict(X_train, y_train, X_val, k=5)
3  nonconformity_scores = torch.abs(y_val - y_val_pred)

```

In this code snippet, nonconformity scores are computed. The function `knn_predict(X_train, y_train, X_val, k=5)` generates predictions for the validation set (`X_val`) using the KNN algorithm, where `k=5` specifies the number of nearest neighbours considered. This function can be defined using the code mentioned in the previous section on K-Nearest Neighbours (3.4.1). The line `nonconformity_scores = torch.abs(y_val - y_val_pred)` computes the nonconformity scores by taking the absolute differences between the true validation values (`y_val`) and the predicted values (`y_val_pred`). These scores put a value to the differences between the model's predictions and the actual values.

To define the prediction intervals, we calculate a quantile from the nonconformity scores. A quantile represents a point in the distribution of data below which a certain percentage of the data falls. For example, the 95th percentile is the value below which 95% of the data lies.

In this case, the quantile is determined by the desired confidence level, denoted as $1 - \alpha$, where α is the significance level. For instance, if we aim for 95% confidence, α would be 0.05, and we would compute the 95th percentile of the nonconformity scores.

The following line of code computes the quantile:

```
1 q = torch.quantile(nonconformity_scores, 1 - alpha)
```

Using this method to compute the quantile, we can generate the prediction intervals for the test data via the following method in Python.

```
1 # Determine the (1 - alpha) quantile
2 alpha = 0.05
3 q = torch.quantile(nonconformity_scores, 1 - alpha)
4
5 # Generate prediction intervals for test data
6 y_test_pred = knn_predict(X_train, y_train, X_test, k=5)
7 prediction_intervals = [(y_test_pred[i] - q, y_test_pred[i] +
8   ↪ q)...
9 ...for i in range(y_test_pred.size(0))]
```

3.7 Parallelisation using CPU

High computational demands can be tackled by parallelisation of the CPU. The `joblib` library of Python very well caters to parallel processing and provides a very robust way of making optimum use of KNN computations.

KNN algorithms are able to be run in parallel in nature: distance calculations between test samples and training samples, and aggregation operations to predict each test sample's class or value. Using `joblib` with the goal of parallelising these operations will significantly reduce the execution time by distributing those operations across multiple CPU cores.

This is one of the most important points when it comes to the `joblib` library: a context manager `parallel_backend` allows running parallel processing by defining which parallel backend to be used with `joblib` parallel functions. The default parallel backend in `joblib` is called `Loky`, and it is designed for efficient handling of CPU-bound tasks. It allows multiprocessing; by launching separate worker processes, `loky` makes sure that tasks are distributed across multiple CPU cores. Further documentation can be found at [8].

Suppose we wish to make some predictions for values on a subset of features using some KNN algorithm. We can optimise the process using `parallel_backend` to process several subsets of features in parallel.

First, the function `knn_predict` calculates predictions for test data based on their k-nearest neighbours. Inside this function we have distance computations and aggregation; these parts can be easily run in parallel. We can refer back to the KNN function we defined in the earlier section. In the the distance matrix computation and nearest-neighbour aggregation are executed for batches of test samples. Each batch is processed sequentially, but these operations can be parallelised to enhance performance. Batching is a method that involves the processing of a dataset by splitting it into smaller subsets of data called batches. This is applicable in scenarios where the dataset is either very huge or computationally very expensive, to help in memory usage and provide computational efficiency. Batch processing of data enables systems to work within hardware limitations, for example, a GPU's memory, and exploits parallelism for faster training or inference.

To integrate parallelisation, we use `parallel_backend` from `joblib` to evaluate the model across different feature subsets concurrently:

```
1 with parallel_backend('loky', n_jobs=-1):
2     for num_features in feature_subsets:
3         selected_features = np.random.choice(X.shape[1],
4         ↪ num_features,...
5         ...replace=False)
6         X_train_subset = X_train[:, selected_features]
7         X_val_subset = X_val[:, selected_features]
8         X_test_subset = X_test[:, selected_features]
```

```

8     mse, rmse, prediction_intervals, y_test_pred =
    ↪     evaluate_model(
9     X_train_subset, y_train, X_val_subset, y_val,
    ↪     X_test_subset, y_test)
10    results[num_features] = {'mse': mse, 'rmse': rmse,...
11    ... 'prediction_intervals': prediction_intervals,...
12    ... 'y_test_pred': y_test_pred}

```

In this example, `parallel_backend` is set to `loky` with `n_jobs=-1`, which utilises all available CPU cores for parallel processing. The function `evaluate_model` refers to a function that is to be built that runs the KNN algorithm and performs conformal prediction to produce the prediction intervals on the results. This function is executed in parallel for each feature subset, allowing simultaneous processing.

In summary, the use of `joblib` and `parallel_backend` for parallelising KNN computations provides noticeable performance benefits. With more available cores, the computational load can be more effectively managed. That being said, we will be able to get stronger results by utilising a GPU instead, which will be tackled in the next section.

3.8 GPU Acceleration with PyTorch

The use of GPU acceleration through PyTorch and CUDA provides a significant computational improvement compared to traditional CPU-based operations. This section delves into how we can implement GPU parallelisation using CUDA instead of CPU parallelisation.

PyTorch allows the model to utilise GPUs, if available, through CUDA, enabling parallel processing. The following line allows the program to select an available GPU, with the benefit of being able to fall back to the CPU should the GPU not be available.

```

1     device = torch.device("cuda" if torch.cuda.is_available()
    ↪     else "cpu")

```

For the `knn_predict()` function, we use PyTorch to compute the distances and find the k-nearest neighbours for each of the test points. This code is a little more complex, so let's consider it in a little more detail:

```

1     distances = torch.cdist(X_test[i:i + batch_size], X_train)
2     knn_indices = torch.topk(distances, k,
    ↪     largest=False).indices

```

First, it computes the pairwise distances with the help of the function `torch.cdist()` between the test data batch and the whole training set. Furthermore, since this is an expensive operation, scaling linearly with the size of the dataset, PyTorch, together with its GPU acceleration, reduces this step time drastically due to the parallel processing capabilities.

The distances are computed, and the *k*-nearest neighbours are obtained based on their indices using the `torch.topk()` function. It is in situations like these that `torch.topk()` efficiently picks the smallest distances by actually doing it on the GPU. It is precisely this kind of functionality, offloading such operations to the GPU, that saves time in performance compared to vanilla NumPy implementations, in which these operations will be done on the CPU, one after another.

3.8.1 Distance Calculation

The `torch.cdist` function in PyTorch efficiently computes pairwise distances between two sets of vectors in a multi-dimensional space.

The general form of `torch.cdist` is as follows:

```
torch.cdist(x1, x2, p=2.0)
```

Here, `x1` and `x2` are input tensors of shapes (N, D) and (M, D) , respectively, where N is the number of vectors in `x1`, M is the number of vectors in `x2`, and D is the dimensionality of each vector.

In the context of CUDA, a tensor is a multidimensional array of data that can be processed in parallel on NVIDIA GPUs.

The optional parameter `p` specifies the distance metric to be used, where the default value is `p=2`, representing the Euclidean (L2) distance. The function outputs a tensor of shape (N, M) , where each element corresponds to the distance between a point in `x1` and a point in `x2`.

In traditional KNN implementations using NumPy, matrix operations, such as distance calculation, are not optimised to be parallelised. In contrast, PyTorch sets up matrix operations on the specialised hardware of a GPU—tensor core units—allowing matrix multiplications and other linear algebra tasks to be conducted at an extraordinary speed compared to being performed on a CPU. Central matrix computations in KNN benefit enormously from this parallelised nature of GPU acceleration.

The following code snippet demonstrates the conversion of data into PyTorch tensors and transferring them to the GPU:

```

1 X_train = torch.tensor(X_train,
    ↪ dtype=torch.float32).to(device)
2 X_test = torch.tensor(X_test,
    ↪ dtype=torch.float32).to(device)
3 y_train = torch.tensor(y_train.values,
    ↪ dtype=torch.float32).to(device)

```

By porting the data onto the GPU, the model fully uses the hardware acceleration coming from CUDA. This approach becomes very useful for computations that require large datasets. For example, in KNN, while calculating the pair distances among data points.

3.8.2 Computation Process

The `torch.cdist` function calculates the Minkowski distance between every pair of vectors in `x1` and `x2`, following the equation:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^D |x_i - y_i|^p \right)^{\frac{1}{p}}$$

Where `x` and `y` represent vectors from `x1` and `x2`, respectively. When $p = 2$, this formula reduces to the Euclidean distance, which is the square root of the sum of squared differences across all dimensions and is what will be used for calculating my distances. When $p = 1$, the Manhattan (L1) distance would be computed instead.

3.8.3 Batching, GPU Optimisation, and Handling Large Datasets

The function `torch.cdist` supports batching; it can compute the distances between multiple vectors at once. In particular, in the case that `x1` and `x2` contain an extra first dimension of size `B`, the batch size, the function returns the distances between the batches corresponding to `x1` and `x2`. The output is then of size `(B, N, M)`, with each batch computed independently, which increases computational efficiency for many sets of data.

One of the major challenges in machine learning is how to deal with huge datasets efficiently. Traditionally, this is done with libraries based on a CPU, like NumPy. This has led to performance bottlenecks—normally because of the limited cores on a CPU and sequential processing, increasing computation time and memory usage. On the contrary, PyTorch exploits CUDA to parallelise the operation on thousands of threads of a GPU, reducing computation time and memory consumption significantly. Tensor ma-

nipulation and matrix operations using PyTorch for large datasets are parallelised across multiple cores, minimising the computational overhead, by distributing tasks like subtraction, exponentiation, summation, and square root extraction. It makes optimal use of the parallelisation resources available. It also brings down the data transfers between the CPU and the GPU to a minimum, which further enhances the performance.

Another factor that helps computational efficiency is batch processing. Since processing all the data points is not feasible for very large datasets, partitioning of data into batches is done. It reduces the number of memory accesses and hence reduces the chances of out-of-memory errors. PyTorch enforces better memory management through parallel processing on the GPU, allowing more significant datasets to be processed by the system. Unified memory and asynchronous transfers between the host and the device guarantee low latency in the data pipeline. Thus, the combination of PyTorch with CUDA allows for better scalability than would a regular NumPy implementation on a CPU, which results in much faster processing for larger datasets. The throughput in execution will thus be much higher for these types of machine learning tasks, such as K-nearest neighbours, significantly reducing computational overhead.

4 Implementation

This section describes implementing our aims into Python scripts we can run. We can expect to have two algorithms, one run on the CPU and one run on the GPU, by the end of this section. Full instructions to running the scripts have been provided in the appendix.

4.1 Setting up the Environment

Setting up a proper development environment is necessary for the development and running of machine learning models. In this project, Visual Studio Code served as my primary code editor, and it was run on a Windows machine. This section will explain how to install VSCode and also set up the CUDA toolkit for GPU acceleration.

4.1.1 Installing VSCode

Visual Studio Code (VSCode) is a lightweight, versatile code editor supporting a wide range of programming languages and extensions. To install VSCode, the following steps were followed:

1. Navigate to the official website of VSCode at <https://code.visualstudio.com/>.
2. Download the installer corresponding to your operating system (Windows or Linux).
3. Run the installer and follow the setup wizard to complete the installation.
4. Once the installation is complete, launch VSCode.
5. Inside the development environment, install the Python extension.

4.1.2 Setting up CUDA

The steps for setting up CUDA are as follows:

1. Check the compatibility of your NVIDIA GPU by visiting the official CUDA website at <https://developer.nvidia.com/cuda-toolkit>. Ensure your GPU is listed under supported hardware.

2. Download the CUDA toolkit installer for your operating system from the same site.
3. Follow the installation instructions specific to your operating system, ensuring that you also install the required NVIDIA drivers during this process.
4. Once the installation is complete, verify the installation by running the command `nvidia-smi` in your terminal. This command should return details about your GPU, confirming that CUDA is correctly installed.
5. Repeat these steps installing the NVIDIA cuDNN (CUDA Deep Neural Network) library.
6. Test the installation is complete by running `nvcc -V` in the command console for VSCode.

To install PyTorch with CUDA support:

1. Open VSCode and load in a blank Python script. Ensure console is enabled.
2. Install PyTorch with the appropriate CUDA version. This can be done by visiting [25] and selecting your correct Python and CUDA configuration.
3. Verify that PyTorch is using CUDA by running:

```
import torch
torch.cuda.is_available()
```

4.1.3 Hardware and Software Used

To recreate the results of this report, it is important to ensure the hardware used matches or is close to that which is used for this report. My algorithms have been run using the follow PC specifications.

Component	Specification
Processor (CPU)	AMD Ryzen 5 5600X (6 Cores)
Graphics Card (GPU)	NVIDIA GeForce RTX 3070 (5888 CUDA Cores)
Memory (RAM)	32 GB DDR4

Table 1: PC Hardware Specifications

4.2 Data Pre-processing Implementation

This section describes the way the steps for data pre-processing were carried out and how the transformations of the data set occurred.

First, the Pandas `dropna()` method can be used to drop all rows containing missing values. The method would also make sure there were no incomplete cases in the dataset. There are no rows with missing values in my dataset, but it is important to note in case this project is replicated with a different dataset that has missing values.

The next step was to convert categorical features into numerical representations. To do this, I first identified the categorical features of the dataset—namely 'make', 'model', 'trim', 'body', 'transmission', 'vin', 'state', 'color', 'interior', and 'seller'. Each of these categorical features had discrete values that needed to be encoded.

To encode these categorical variables, I've used the `LabelEncoder` class from the `sklearn.preprocessing` module. We first create a dictionary of label encoders for each of the categorical columns. It thus allows me, if ever the need arise, to later on re-apply inverse transformations to bring back the original labels from the numeric values. For each feature in the list I used `LabelEncoder` and applied it to the corresponding feature in the Data-Frame, while keeping the instance of the fitted encoder. It means basically that each unique category was transformed into a unique integer.

It is wise to also confirm that the columns 'year', 'condition', 'odometer', and 'mmr' are numeric variables. The `pd.to_numeric()` function casts each column to a valid column type for numerical data, automatically handling any possible inconsistencies. This is useful to note as it covers tracks in case we missed a identifying a column that needs encoding.

Then, having encoded the categorical variables and done the conversion of the relevant columns to data types compatible with numeric data, I defined the features (X) and the target variable (y). The features set X was defined by excluding the columns 'sellingprice' and 'saledate' from the Data-Frame. The target variable is chosen to be 'sellingprice' since that is the price the model is to predict, while 'saledate' is removed since it has

no direct effects on the prediction; including the variable might introduce undesirable temporal bias. Therefore, the feature matrix X can include all other variables that can be used to make a prediction about the selling price.

Afterwards, I split the dataset into a training, validation, and testing sets. The data was split into 60% for training and 40% for validation and testing using `'train_test_split'`. The 40% was then equally divided into validation and testing sets. This approach ensures consistent and reproducible splits of the data.

Finally, I standardised the features to normalise them. For this, I used `'StandardScaler'` from `'sklearn.preprocessing'`. I fit the scalar to the training data using `fit_transform()`, to ensure the scaling parameters (mean and variance) would be computed based only on the training set.

The processed data was now at the point where all the categorical features could be encoded, the numerical features standardised, and the datasets suitably split into training, validation, and test sets.

4.3 KNN Prediction

The `knn_predict` function implements K-Nearest Neighbours to make predictions of target values by finding the k -nearest neighbours from a training set. There are two variations of this implementation: one using PyTorch and another one using NumPy. I showed the NumPy implementation earlier in section 3.4.1.

4.3.1 PyTorch Implementation

```
1 def knn_predict(X_train, y_train, X_test, k, batch_size=1024):  
2     y_pred = []
```

The function begins by initialising an empty list, `y_pred`, which will store the predictions for each test data batch.

```
1     for i in range(0, X_test.size(0), batch_size):
```

This loop processes the test data in batches, as processing the entire test set at once might be computationally expensive. The loop index, `i`, moves in steps of `batch_size` to process test points in batches.

```
1         distances = torch.cdist(X_test[i:i + batch_size],  
                                  ↪ X_train)
```

Here, `torch.cdist` calculates the Euclidean distance between the current test batch `X_test[i:i + batch_size]` and the entire training set `X_train`.

```
1 knn_indices = torch.topk(distances, k,  
    ↪ largest=False).indices
```

`torch.topk` finds the indices of the k smallest distances for each test point in the batch, giving the k nearest neighbours from the training set. The `largest=False` argument ensures that the smallest distances are selected.

```
1 batch_y_pred = y_train[knn_indices].mean(dim=1)
```

This line predicts the target value for each test point by averaging the target values (`y_train`) of the k nearest neighbours. The mean is computed along dimension 1, which corresponds to the k nearest points.

```
1 y_pred.append(batch_y_pred)  
2 return torch.cat(y_pred)
```

The predicted values for the current batch are appended to the `y_pred` list, then the list of batch predictions is concatenated into a single tensor and returned.

4.3.2 NumPy Implementation

The NumPy implementation begins the same. We define the function and initialise an empty list. Then begin the loop using batches.

```
1 distances = np.linalg.norm(X_test[i:i + batch_size,  
    ↪ np.newaxis] - X_train, axis=2)
```

Instead of using `torch.cdist`, the NumPy implementation calculates Euclidean distances using `np.linalg.norm`. The batch of test points `X_test[i:i + batch_size]` is expanded to three dimensions using `np.newaxis`, allowing broadcasting to compute distances from all training points `X_train`.

```
1 knn_indices = np.argsort(distances, axis=1)[: , :k]
```

Here, `np.argsort` sorts the distances for each test point along axis 1. The indices of the k smallest distances are selected using slicing (`[ : , :k]`),

representing the nearest neighbours.

```
1 batch_y_pred = np.mean(y_train[knn_indices], axis=1)
```

The predictions are made by averaging the target values (`y_train`) of the k nearest neighbours, as done in the other implementation.

```
1 y_pred.append(batch_y_pred)
```

The predicted values for the current batch are appended to `y_pred`.

```
1 return np.concatenate(y_pred)
```

Finally, the list of batch predictions is concatenated into a single array and returned as the output of the function.

The implementations only differ by a tiny bit structurally: the test data is processed batch by batch, distances are computed, k nearest neighbours are picked out, and target values are predicted by taking the average of the target values of the neighbours. Specifically, the PyTorch version uses computation on a GPU for efficiency with `torch.cdist` for distances, and the NumPy version uses CPU computation with `np.linalg.norm` for the distances.

4.4 Conformal Prediction Interval Calculation

```
1 # Calculate nonconformity scores on the calibration set,  
  ↪ the absolute difference between the actual and  
  ↪ predicted values  
2 # Nonconformity scores measure how "different" or  
  ↪ "nonconforming" the predicted values are from the  
  ↪ actual values  
3 nonconformity_scores = np.abs(y_val - y_val_pred)
```

The first line calculates the absolute error between the predicted values for the validation set, `y_val_pred`, and the actual values, `y_val`. In Python, this will be done using the NumPy function `np.abs`. More precisely, this function written above returns an array of element-wise absolute differences between the true and predicted values; these are the nonconformity scores.

```
1 # Determine the (1 - alpha)-quantile of the nonconformity  
  ↪ scores, percentile of 95%
```

```

2     alpha = 0.05
3     q = np.quantile(nonconformity_scores, 1 - alpha)

```

Next, compute the $(1 - \alpha)$ quantile of the nonconformity scores. Here it makes use of a variable α of 0.05, which is the equivalent of a 95% level of confidence. This quantile is used as regards prediction intervals to encapsulate the degree of error from the model's predictions. Here, the `np.quantile()` is called to compute this quantile; that will be the value below which a specified proportion of the nonconformity scores fall. Thus, it will define the threshold that will later be used in creating the prediction intervals.

```

1     # Predict on the test data
2     y_test_pred = knn_predict(X_train, y_train, X_test, k)

```

The third line calls the `knn_predict()` function again to generate the predictions for the test data.

```

1     # Generate prediction intervals for the test data
2     prediction_intervals = [(y_test_pred[i] - q, y_test_pred[i]
    ↪ + q) for i in range(y_test_pred.shape[0])]

```

Then the prediction intervals of the test data using the quantile q calculated above are constructed. This line `[(y_test_pred[i] - q, y_test_pred[i] + q) for i in range(y_test_pred.shape[0])]` iterates over all of the predicted values. For each prediction i , the lower bound of the interval is computed by the prediction minus the quantile q , and the upper bound by addition. It effectively creates a symmetric prediction interval around each prediction, designed to contain the true target value at a probability of 95%.

We stuck with the NumPy versions of two specific lines when generating the intervals. These lines calculated the absolute value and calculated the quantiles, and can similarly be done in PyTorch like below.

```

1     # Absolute value with PyTorch
2     nonconformity_scores = torch.abs(y_val - y_val_pred)
3     # Quantile calculation with PyTorch
4     q = np.quantile(nonconformity_scores, 1 - alpha)

```

4.5 GPU and CPU Performance

Only the 2-feature model was tested for the CPU version because of runtime considerations. KNN computations on large datasets require the calculation of pairwise distances between data points, which can become computationally very expensive as the number of features increases. Due to high memory usage and extended run times, it is not practical to test higher-dimensional models on a CPU, and we can expect the runtime for more than 2 features to be very high.

We can control this between the two algorithms by wrapping the main function of the algorithm in a loop. We can use `for num_features in feature_subsets:`, which runs the algorithm a number of times equal to the number of features in the selected feature subset. That subset is defined beforehand as a dictionary like so:

```
1 feature_subsets = {  
2   2: ['odometer', 'mmr']  
3 }
```

On the other hand, thanks to parallel computation in a GPU, handling large-scaled matrix operations more efficiently than a CPU, we could test models with up to 7 features. We can expect significant improvements in runtime using CUDA allowing us to use more features with the model.

We can also make predictions on how long the CPU program will take to run with more features based off the ratio we can infer from the runtime comparison with 2 features.

4.6 Evaluation metrics for the algorithm

We discussed that MSE and RMSE will be the performance indicators for the models. These can be calculated using the `sklearn.metrics`, specifically the `mean_squared_error` function. Below is the function as defined by the library.

```
1 (function) def mean_squared_error(  
2   y_true: MatrixLike | ArrayLike,  
3   y_pred: MatrixLike | ArrayLike,  
4   *,  
5   sample_weight: ArrayLike | None = None,  
6   multioutput: ArrayLike | Literal['raw_values',  
   ↪ 'uniform_average'] = "uniform_average",
```

```
7     squared: bool = True
8 ) -> (Float | ndarray)
```

Inside KNN loop, we can call the function and then return the MSE with the model predictions. The same can be done with the RMSE, which is a simple calculation as it is just the square root. Square root calculations are made easy with the NumPy library.

```
1     mse = mean_squared_error(y_test, y_test_pred)
2     rmse = np.sqrt(mse)
```

5 Results

In this section, we will expect to see graphs of results from the KNN model with the resulting prediction intervals for each estimate. We also will see a comparison of non-conformity scores and runtime differences between GPU and CPU. The model is run from 2 to 7 features.

5.1 MSE, RMSE and coverage probability for Different Feature Sets on GPU

We look at three main metrics when evaluating our conformal prediction KNN model: the Mean Squared Error (MSE), the Root Mean Squared Error (RMSE), and the Coverage Probability. MSE gives the average of squared differences between the predicted and real values, while RMSE quantifies the error in the same units as the target variable itself, hence providing a more interpretable metric. Coverage probability, on the other hand, measures the proportion of true values that fall within the prediction interval, reflecting the reliability of the model's uncertainty estimates.

The table below shows the Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Coverage Probability for my KNN model implemented with GPU acceleration, evaluated using 2, 3, 4, 5, 6, and 7 features.

Table 2: MSE, RMSE, and Coverage Probability for KNN Model Using 2 to 7 Features (GPU)

Number of Features	MSE	RMSE	Coverage Probability
2 Features	3534796.00	1880.1053	0.9499
3 Features	2669760.75	1633.9403	0.9506
4 Features	2677677.25	1636.3610	0.9508
5 Features	2729124.25	1652.0061	0.9511
6 Features	2751474.25	1658.7568	0.9505
7 Features	2910394.75	1705.9879	0.9508

5.1.1 CPU MSE/RMSE

MSE	RMSE
4655241.13	2157.60

Table 3: CPU MSE Results

5.2 Graphs of predictions for each feature set

Figure 1 is a scatter graph showing the true values, the predicted values and the prediction interval for each label from the KNN model, running for 2 features on the CPU. We picked a sample of 100 data points to display for better legibility of the graph. This is repeated for figures 2, 3, 4, 5, 6 and 7, which show the same data run on the GPU instead with increasing features (2-7 inclusive respectively).

5.2.1 CPU Acceleration

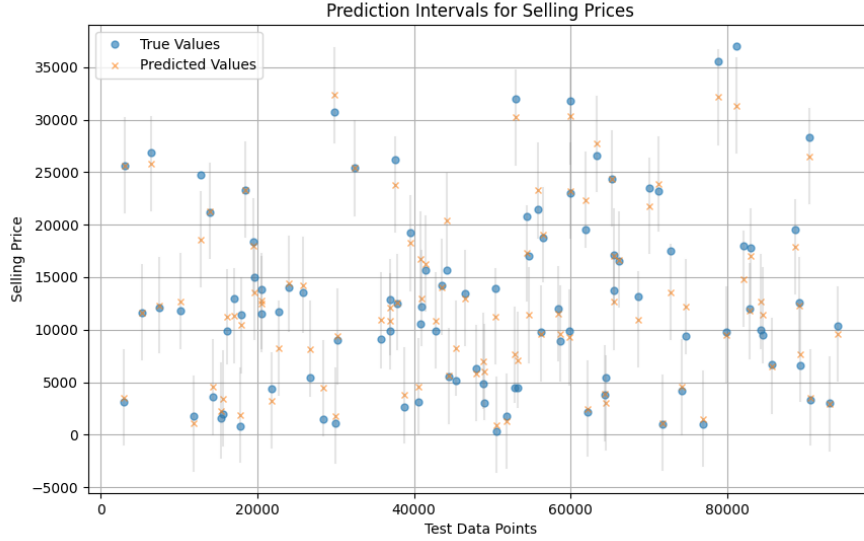


Figure 1: Prediction Intervals for CPU Performance

5.2.2 GPU Acceleration

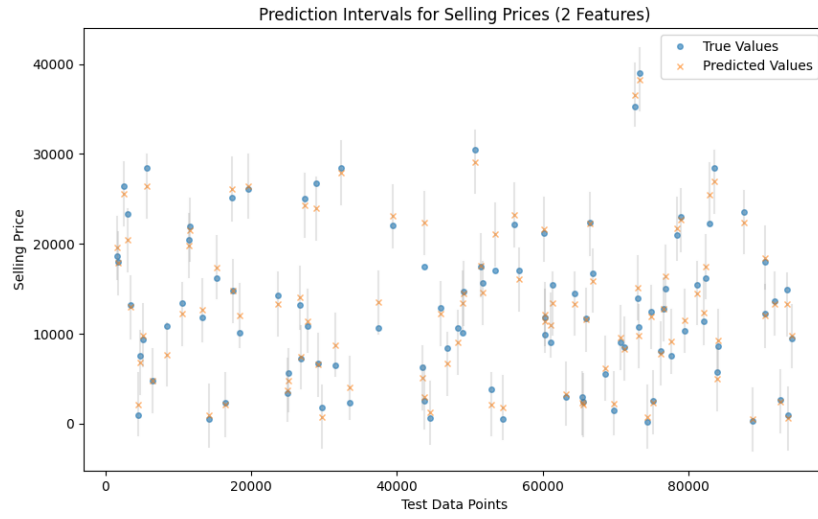


Figure 2: Prediction Intervals for GPU Performance with 2 Features

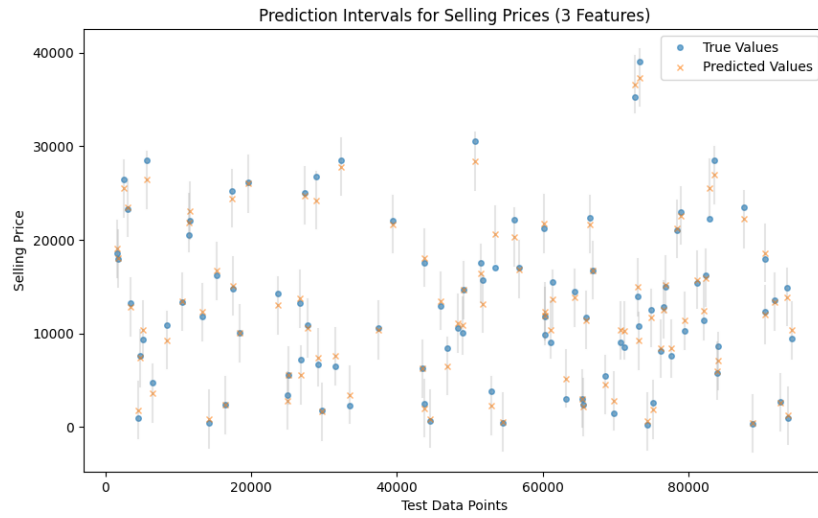


Figure 3: Prediction Intervals for GPU Performance with 3 Features

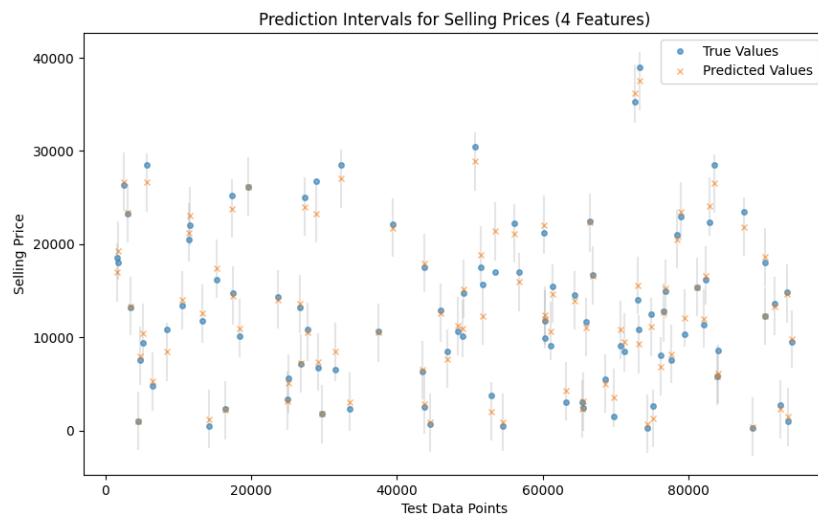


Figure 4: Prediction Intervals for GPU Performance with 4 Features

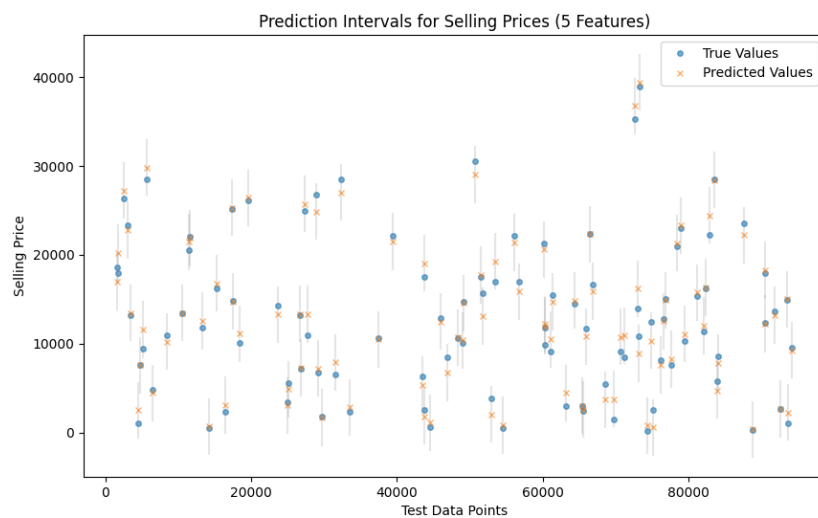


Figure 5: Prediction Intervals for GPU Performance with 5 Features

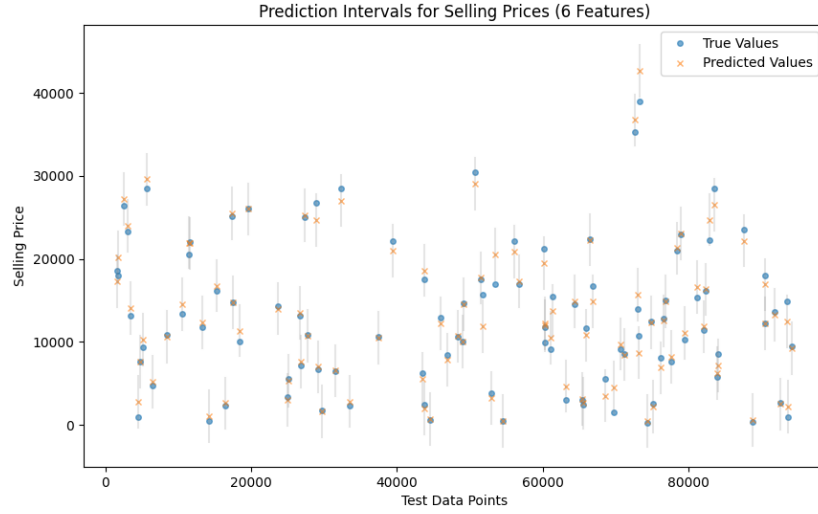


Figure 6: Prediction Intervals for GPU Performance with 6 Features

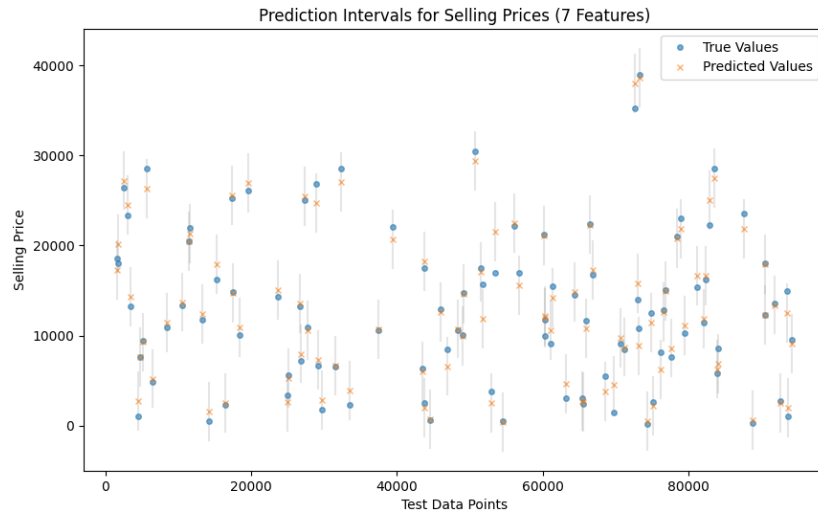


Figure 7: Prediction Intervals for GPU Performance with 7 Features

5.3 CDF of Non-Conformity Scores for GPU models

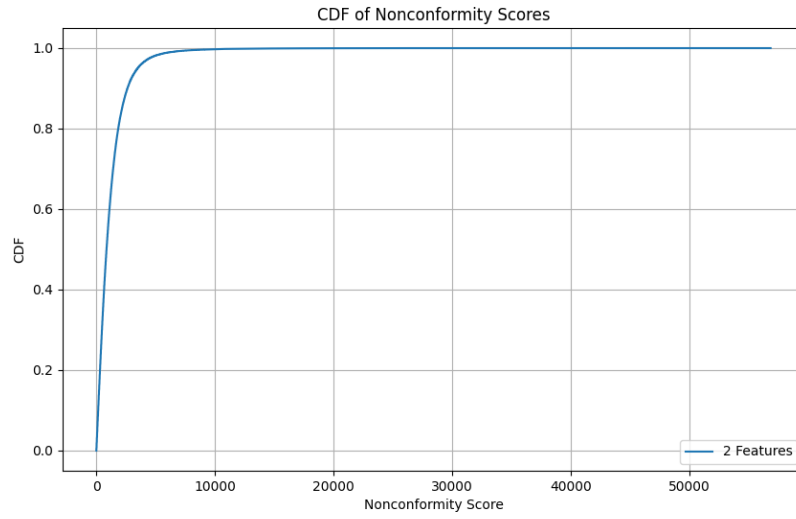


Figure 8: Nonconformity Scores for 2 Features

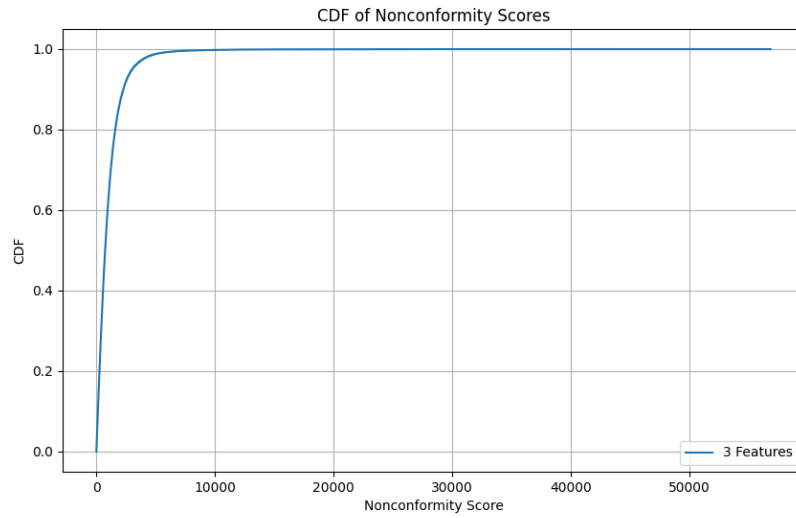


Figure 9: Nonconformity Scores for 3 Features

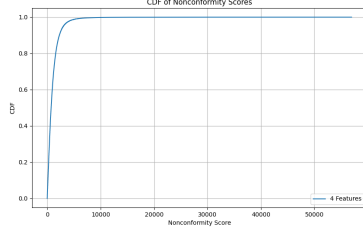


Figure 10: Nonconformity Scores for 4 Features

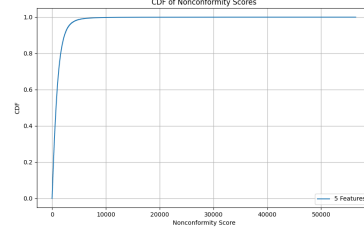


Figure 11: Nonconformity Scores for 5 Features

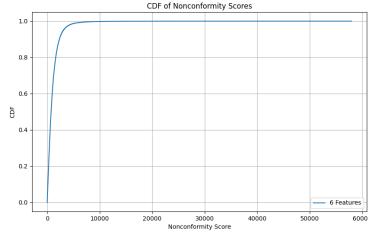


Figure 12: Nonconformity Scores for 6 Features

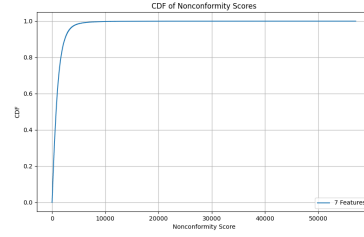


Figure 13: Nonconformity Scores for 7 Features

5.4 Performance Analysis

5.4.1 Runtime Comparison between GPU and CPU

No. of Features	CPU Runtime (seconds)	GPU Runtime (seconds)
2	5760.3287	7.7955
3	N/A	8.6758
4	N/A	7.8915
5	N/A	7.9411
6	N/A	8.0130
7	N/A	8.1013

Table 4: Runtimes for KNN Conformal Prediction with 2 to 7 Features (CPU vs GPU)

6 Discussion

Here we aim to explore and compare the results with the methods and expectations we discussed earlier.

6.1 Interpretation of Results

For all models generated, we can see that from the sample size we have, the predictions tend to be generally quite close to the actual results. This is supported by the coverage probability for the 6 GPU models being above 95%. Most actual values fall within the prediction and the prediction intervals, rarely falling outside of the range. The accuracy does vary slightly between models, and this can usually be for a number of reasons, but here is likely due to the augmentation of features used in the model. In general, we do not see a visible improvement with adding more features to the model, which suggests that if we were to increase the dimensionality of the model further to make the model even more accurate, then it would not be viable.

This is further supported by the minimal changes in the model RMSE. In general, the lower the RMSE, the better the model performs. The problem with MSE is that the actual number is very contextual, whether it shows whether the model is good or not. What we can infer from the RMSE values is that the model with 2 features is the most performing, as in the predicted values are the closest to the actual values. Other models may perform worse with the addition of extra features due to those features adding noise, which refers to unwanted or irrelevant information that can interfere with the accuracy and reliability of the data.

The figures 8 through to 13 show the cumulative distribution functions of the nonconformity scores obtained on the different feature sets with sizes ranging from 2 to 7 features. These nonconformity scores measure how well model predictions agree with actual outcomes for every data point. A lower nonconformity score means that the model is doing well on that sample, its prediction being closer to the true value. In contrast, the higher the score of nonconformity, the larger the quantity of deviation from the model's proposition of a prediction.

In all the graphs, there is a steep increase in the lower values of the nonconformity score. A steep rise indicates that most data points will have low scores of nonconformity, which implies these data points are well-conforming to the model's predictions. This would further suggest that for most samples, the model generally performed well and gave correct predictions. As the graphs ended up being so similar, for space concerns, Figures 10 to 13

are condensed, though they largely show the exact same trend shown in Figure 9.

When the sum of nonconformity scores increases, the curve of CDF increases and levels off, indicating a smaller proportion of samples with high scores of nonconformity. High nonconformity scores refer to samples where the model’s predictions deviate substantially from the actual outcomes, therefore indicating poorer performance on such data points. If the CDF levels out at these higher scores, it means that only a small fraction of the data points are considered this class of high nonconformity; therefore, most predictions are reasonably sensible.

This means that adding more features does not seriously impact the distribution of the nonconformity score, which, in other terms, might mean that adding more features to the model does not necessarily affect the conformity of the predictions. The stability of the distribution for the various sets of features would then suggest that the robustness of generalisation of predictions by the model remains strong, even with an increase in the number of features.

To summarise, the above CDF graphs of the nonconformity score, the model shows good performance on most samples, except for a few outliers. Besides, the consistency in the distribution of these nonconformity scores across different sets of features points to the fact that the predictive performance of the model does not suffer radical changes even when new features are added to the set. This may imply a stability of the model where a relationship is captured accordingly in the data without overfitting, or perhaps it suffers less from diminishing returns when incorporating more features.

The runtime of the model differs significantly when comparing which method of parallelisation was used. We did expect there to be a noticeable difference, yet the sheer difference in performance was still very impressive. The reason for this is due to the number of cores available on the GPU compared to the CPU (5888 CUDA cores on the GPU vs 6 CPU cores). Using the values we have in table 3, we can attempt to provide a rough estimate of runtime using our existing runtimes.

Calculate the CPU-to-GPU Ratio for Feature 2

For Feature 2, the runtimes (to 2 decimal places) are:

$$\text{CPU time} = 5760.33 \quad \text{GPU time} = 7.80$$

The CPU-to-GPU runtime ratio is:

$$\text{Ratio} = \frac{5760.33}{7.8} = 738.50$$

Estimate CPU Runtimes for Features 3 to 7

Using this ratio, we estimate the CPU runtimes for Features 3 to 7:

$$\text{Feature 3 CPU Runtime} = 8.68 \times 738.50 = 6410.18$$

$$\text{Feature 4 CPU Runtime} = 7.89 \times 738.50 = 5826.77$$

$$\text{Feature 5 CPU Runtime} = 7.94 \times 738.50 = 5863.69$$

$$\text{Feature 6 CPU Runtime} = 8.01 \times 738.50 = 5915.39$$

$$\text{Feature 7 CPU Runtime} = 8.10 \times 738.50 = 5981.85$$

These times range from approximately 97 to 107 minutes. Due to the high resource demands of the algorithm, with each run taking between 97 and 107 minutes and placing significant heat stress on my CPU, I opted to run the analysis once to obtain a representative result. Repeated runs were deemed unnecessary, as we can estimate CPU runtimes using the GPU results and the calculated CPU-to-GPU ratio of 738.50, as demonstrated in the calculations above.

Interestingly, the GPU runtimes did not follow the simple linear increase that was initially expected. The runtime for the 2-feature model was 7.80 seconds, but this spiked unexpectedly to 8.68 seconds for the 3-feature model before dropping back to 7.89 seconds for 4 features. From that point on, a small, incremental increase in computation time was observed, with the runtime only rising to 8.10 seconds for the 7-feature model. This indicates that while adding more features does increase computational overhead, the effect is minimal on a massively parallel architecture like a GPU. Therefore, it was surprising to observe the stability of the performance, with the exception of the anomalous result for the 3-feature set, which could be attributed to factors such as memory access patterns or simple measurement variance, especially since these tests were run on a home personal computer where other applications could have been utilising the GPU.

6.1.1 Summary of Results

The KNN models demonstrated satisfactory performance, accurately predicting the majority of the values. The addition of features did not substantially alter the model's performance. Counter-intuitively, GPU runtime decreased with the inclusion of features 3 to 7, suggesting computational efficiency even with increased dimensionality. The significant computational time ratio between CPUs and GPUs underscored the superior performance

of GPUs attributed to their substantial number of cores. The actual ratio in computational time between CPUs and GPUs proved to be great in terms of runtime performance, proving that indeed a GPU had higher performance due to its huge number of cores.

6.2 Practical Implications

As we mentioned previously, there is no strict need to stick with the cars data set we used for this project. The benefit of the techniques used here (KNN, conformal prediction and CUDA based GPU Acceleration) are that they are extremely versatile and can be utilised to analyse a variety of different datasets.

The most significant advantage highlighted is the role of GPU acceleration in significantly reducing prediction times for models such as KNN. In sectors where vast amounts of data need to be processed quickly, like finance, health care, and e-commerce, using GPU power to speed up machine learning tasks can facilitate massive real-world improvements. This capability enables systems to provide instant feedback or predictions, which is essential in environments that require rapid decision-making, such as fraud detection or personalised recommendations in healthcare. This is where traditional methods of data analysis or even CPU-based models tend to collapse, as the speed will not make the cut. Being able to process more data with greater speed allows necessary fields to be efficient and corporate companies to stay competitive.

6.3 Limitations of the Study and Future Work

This project was not without difficulty. The main being capturing the full scope of the possibilities of the project within the given time frame. This section aims to highlight the key limitations within the project and quickly discuss the methods at which they could be implemented in the future given ample time.

- **Complexity of the CPU algorithm using joblib:** The primary goal of this study was the use of CUDA to perform GPU based hardware acceleration, so the inclusion of learning joblib and understanding how to implement CPU-based parallelisation was an attempt to go beyond the scope of the initial project. Whilst it made a good point of comparison, it ultimately shows a weak point in the study as other areas are not as strong as they could have been due to time constraints.

- **Proper integration of CPU performance evaluation:** The inclusion of CPU-based conformal prediction using KNN in this study is very bare-bones. The reason for this is due to the extensive time required to run KNN on large datasets such as the one chosen for this project. With more time available with the working algorithm, it would have been a welcome addition to run the algorithm with more features and make further additions similar to the ones I would want to add to the GPU edition of the program, which are mentioned below.
- **Enhanced model performance:** The model performance is measured by the MSE/RMSE. For four models produced, these values are extremely high. Luckily the need for a highly performing model was minimal for this project; nevertheless having a highly performing model can prove useful should the work here be replicated for another dataset of more importance. There are some methods one can employ to improve the model performance. These include optimising the value for K, using different methods for measuring distance and hyperparameter tuning. We can expect all these methods to give a change to the MSE of the models. Further reading on these can be found at [23, 18].
- **Usability optimisations:** The algorithms provided in this research all primarily focus on providing a prediction with the corresponding interval. The problem here is that it is limited to providing predictions strictly on the data inside the dataset. It would prove to be useful if there was the capability to user input feature data and the model to attempt to predict the value of the car.

6.4 Personal Assessment

It was clear early on that for this project I had to be decisive about what subjects to focus on. The topic title being GPU-Accelerated conformal prediction implies splitting the project into 3 sections. These are machine learning algorithms, conformal prediction and GPU parallelisation. Knowing this, when the project began I was able to setup a project plan to attempt to dictate goals and determine how much time to distribute across these 3 aspects and also take into consideration the time to write the algorithms and the report itself.

The first hurdle to overcome was wrapping my head around the idea of conformal prediction. The paper by Vovk and Shafer [9] gave me a foothold as I had not covered much if anything to do with the topic at all so far in my

studies. With the help of this resource, and the reading of further studies into various machine learning algorithms that I had already covered in my course, I decided to settle on KNN as my choice of algorithm and I can build conformal predictors on the estimates produced by the KNN models.

Python was my language of choice, as I have the most experience with data handling in this language. The other choice was R, and despite having written ML algorithms in R in previous studies, Python was the obvious choice here due to the vast libraries available to perform GPU acceleration. Now having completed this project, I would not have picked a different language for it.

The next problem was getting CUDA to be functional on my home PC. I am fortunate to have an NVIDIA Graphics Card that supports CUDA, which is also the reason why I decided to pick this project, but I did not anticipate the difficulty of getting CUDA working within my VSCODE workspace. The main problem being able to install RAPIDS, which is suite of libraries to help with GPU acceleration and is what is recommended for anything related to data analytics or ETL (Extract, Transform, Load). Being wary of time constraints, I had to opt for an alternative. I eventually settled on PyTorch which was still more than capable of running everything I needed to do.

Once I had gotten GPU acceleration successfully working, the next step was to start coding, and my goal was to have all programs complete before starting any of the writing. This meant that when I do begin the report I'll have a clear end point of what I need to write towards, what relevant literature to include and how the project should generally look in comparison to my algorithms. This step was challenging, as I had to work with two libraries I have never seen before. These being joblib for CPU acceleration and PyTorch for GPU. Nevertheless it was what I expected when I chose the topic and in hindsight I should have started learning the libraries earlier to cut down on the time spent writing the python scripts.

The final step was writing the report, and this was perhaps the part I struggled with most. It is always difficult to predict how a report is going read once you have finished writing it, and coherency is one of the most important aspects in making a report engaging.

Overall, I am confident in the accuracy of the definitions in this report and in the findings of the performance improvements utilising GPU. The scripts submitted alongside the report are well commented throughout, making them highly usable for further studies.

References

- [1] Pandas documentation. https://pandas.pydata.org/docs/user_guide/missing_data.html. Accessed: June 2024.
- [2] Sklearn preprocessing documentation. <https://scikit-learn.org/stable/modules/preprocessing.html>. Accessed: June 2024.
- [3] Naomi S. Altman. An introduction to kernel and nearest-neighbor non-parametric regression. *The American Statistician*, 46(3):175–185, 1992.
- [4] Gene M Amdahl. Validity of the single processor approach to achieving large-scale computing capabilities. *AFIPS spring joint computer conference*, pages 483–485, 1967.
- [5] Anwar. Vehicle sales data. <https://www.kaggle.com/datasets/syednwarafриди/vehicle-sales-data/discussion?sort=hotness>, 2024. Accessed: June 2024.
- [6] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006.
- [7] John Daintith and Edmund Wright. *A Dictionary of Computing*. Oxford University Press, 6 edition, 2008.
- [8] Joblib Developers. `Joblib.parallel`, 2024. Accessed: 2024-08-15.
- [9] Vladimir Vovk Glenn Shafer. A tutorial on conformal prediction. 2008.
- [10] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.
- [11] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3:1157–1182, 2003.
- [12] John L Hennessy and David A Patterson. *Computer architecture: a quantitative approach*. Morgan Kaufmann, 2019.
- [13] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.

- [14] Institute for the Advancement of University Learning and Department of Statistics. *Descriptive Statistics for Research*. Institute for the Advancement of University Learning, Hilary Term, 2002. Lecture 5: Confidence Intervals.
- [15] Norman P Jouppi, Cliff Young, Nishant Patil, David A Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, and Al Borchers. In-datacenter performance analysis of a tensor processing unit. *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12, 2017.
- [16] David B Kirk and Wen-mei W Hwu. *Programming massively parallel processors: a hands-on approach*. Morgan kaufmann.
- [17] Daren Lee, Ivo Dinov, Bin Dong, Boris Gutman, Igor Yanovsky, and Arthur W. Toga. CUDA Optimization Strategies for Compute- and Memory-Bound Neuroimaging Algorithms. 2010.
- [18] Masoud Maleki, Negin Manshouri, and Temel Kayıkçıoğlu. A Novel Simple Method to Select Optimal k in k-Nearest Neighbor Classifier. 2017.
- [19] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.
- [20] Sparsh Mittal and Jeffrey S. Vetter. A Survey of CPU-GPU Heterogeneous Computing Techniques. *ACM Computing Surveys (CSUR)*, 2015.
- [21] John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. Scalable parallel programming with cuda. *Queue*, 6(2):40–53, 2008.
- [22] NVIDIA. Cuda c++ programming guide. 2022.
- [23] Rukshan Pramoditha. Choosing the Right Number of Neighbors (k) for the K-Nearest Neighbors (KNN) Algorithm, 2024. Accessed: August 2025.
- [24] J. Serrano. Introduction to FPGA design. 2008.
- [25] PyTorch Team. *PyTorch Installation Instructions on Windows*, 2024. Available at: <https://pytorch.org/get-started/locally/>.
- [26] Kevin Vincent, Damien Nguyen, Brian Walker, Thomas Lu, and Tien-Hsin Chao. GPU processing for parallel image processing and real-time object recognition. 2014.

A A: How to run the Python scripts

This appendix provides guidance on running my Python scripts for this project. The instructions assume all necessary dependencies are installed, as detailed in the implementation section in the report.

A.1 Prerequisites

Ensure the following libraries are installed:

- Python (v3.7 or higher)
- PyTorch (with CUDA and cuDNN for GPU acceleration)
- Microsoft VSCode

Install the dependencies using:

```
pip install (package_name)
```

For GPU support, verify PyTorch is using CUDA:

```
import torch
print(torch.cuda.is_available())
```

Remember that to run the GPU implementations, a CUDA-supported graphics card is required. A comprehensive list can be found on NVIDIA's site.

Finally, ensure the correct dataset has been downloaded from the link in the bibliography. This should be placed in the same directory as the Python scripts.

A.2 Running the Scripts

1. Run VSCode, ensuring all dependencies have been installed.
2. Run the desired Python script, e.g.:

```
python CPU Script.py
```